

图 3.4 **id * id * id** 和 **id+id * id** 的分析树

到的分析树就不能很方便地用于语义分析和中间代码生成等阶段。例如,如果规定 * 和+是右结合的运算,那么文法应该如下:

$$expr \rightarrow term + expr \mid term$$

$$term \rightarrow factor * term \mid factor$$

$$factor \rightarrow \mathbf{id} \mid (expr)$$

比较图 3.5 和图 3.4 的分析树,应该不难看出它们的区别。

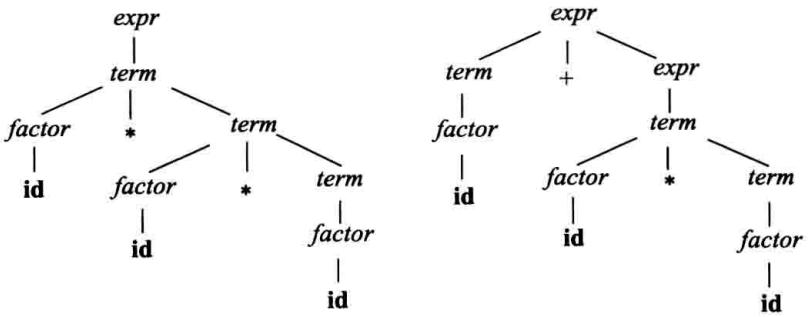


图 3.5 文法修改后的分析树

3.2.5 消除二义性

从上一小节的例子可以看到,有些二义文法可以通过重写文法而消除二义性。再举一个例子,消除下面“悬空 else”文法的二义性:

$$\begin{aligned} stmt \rightarrow & \mathbf{if}\ expr\ \mathbf{then}\ stmt \\ & \mid \mathbf{if}\ expr\ \mathbf{then}\ stmt\ \mathbf{else}\ stmt \\ & \mid \mathbf{other} \end{aligned} \tag{3.5}$$

这里的 **other** 代表任何其他语句。按照这个文法,形式为

$$\mathbf{if}\ expr\ \mathbf{then}\ \mathbf{if}\ expr\ \mathbf{then}\ stmt\ \mathbf{else}\ stmt \tag{3.6}$$

的嵌套条件语句有两个最左推导:

$$stmt \Rightarrow \mathbf{if}\ expr\ \mathbf{then}\ stmt \Rightarrow \mathbf{if}\ expr\ \mathbf{then}\ \mathbf{if}\ expr\ \mathbf{then}\ stmt\ \mathbf{else}\ stmt$$

stmt ⇒ **if** *expr* **then** *stmt* **else** *stmt* ⇒ **if** *expr* **then if** *expr* **then** *stmt* **else** *stmt*

因此文法(3.5)是二义的。

所有含这种条件语句的语言都使用前一种最左推导,因为它和这些语言所采用的规则"每个 **else** 和左边最接近的还没有配对的 **then** 相配对"是一致的。这条规则可以直接体现在文法中,例如,可以把文法(3.5)改写成下面无二义的文法。想法是这样的,出现在 **then** 和 **else** 之间的语句必须是"配对"的,配对语句是指那些不是条件语句的语句,还有那些只含配对语句的 if-then-else 语句。

$$
\begin{aligned}
&stmt \rightarrow matched_stmt \\
&\quad\ \mid unmatched_stmt \\
&matched_stmt \rightarrow \textbf{if}\ expr\ \textbf{then}\ matched_stmt\ \textbf{else}\ matched_stmt \\
&\quad\ \mid \textbf{other} \qquad (3.7) \\
&unmatched_stmt \rightarrow \textbf{if}\ expr\ \textbf{then}\ stmt \\
&\quad\ \mid \textbf{if}\ expr\ \textbf{then}\ matched_stmt\ \textbf{else}\ unmatched_stmt
\end{aligned}
$$

文法(3.7)和文法(3.5)产生同样的串集,但是对于句型(3.6)只存在一种最左推导。注意,*unmatched_stmt*第二个产生式的右部 **if** *expr* **then** *matched_stmt* **else** *unmatched_stmt* 的 *matched_stmt* 和 *unmatched_stmt* 是不能对调的,否则仍然是二义的。

也许你会问,为什么大多数编程语言都不用无二义的文法(3.7),而采用二义文法(3.5)呢?这是因为,文法(3.7)失去了简洁性。定义语言语法的文法有二义性并不可怕,只要有消除二义性的规则就可以了。

3.2.6 消除左递归

一个文法是**左递归的**,如果它有非终结符 A,对某个串 α,存在推导 $A \Rightarrow^{+} A\alpha$。自上而下的分析方法不能用于左递归文法,因此需要消除左递归。由形式为 $A \rightarrow A\alpha$ 的产生式引起的左递归称为**直接左递归**。

左递归产生式 $A \rightarrow A\alpha \mid \beta$ 可以用非左递归的

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha \mid A' \mid \varepsilon$$

来代替,它们没有改变从 A 推导出的串集。

例 3.6 考虑下面的算术表达文法:

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \textbf{id}$$

消除 E 和 T 的直接左递归,可以得到

$$E \rightarrow TE'$$

$$
\begin{aligned}
&E' \rightarrow +TE' \mid \varepsilon \\
&T \rightarrow FT' \\
&T' \rightarrow *FT' \mid \varepsilon \\
&F \rightarrow (E) \mid \mathbf{id}
\end{aligned}
\tag{3.8}
$$

□

不管有多少 A 产生式，都可以用下面的技术消除直接左递归。首先把 A 产生式组合在一起：

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

其中 β_i 都不以 A 开始，α_i 都非空，然后用

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$$

代替 A 产生式。这些产生式和前面的产生式产生一样的串集，但是不再有左递归。这个过程可删除直接左递归，但不能消除两步或多步推导形成的左递归。例如，考虑文法

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Sd \mid \varepsilon$$

其中非终结符 S 是左递归的，因为 $S \Rightarrow Aa \Rightarrow Sda$，但它不是直接左递归的。用 S 产生式代换 $A \rightarrow Sd$ 中的 S，可以得到下面的文法：

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Aad \mid bd \mid \varepsilon$$

删除其中的直接左递归，得到如下的文法：

$$S \rightarrow Aa \mid b$$

$$A \rightarrow bdA' \mid A'$$

$$A' \rightarrow adA' \mid \varepsilon$$

由此可见，写一个删除文法左递归的算法并不是件困难的事情。

3.2.7 提左因子

提左因子也是一种文法变换，它用于产生适合于自上而下分析的文法。在自上而下的分析中，当不清楚应该用非终结符 A 的哪个选择来替换它时，可以通过重写 A 产生式来推迟这种决定，推迟到看见足够多的输入，能帮助正确决定所需选择为止。

例如，条件语句有两个产生式：

$$
\begin{aligned}
stmt \rightarrow &\mathbf{if}\ expr\ \mathbf{then}\ stmt\ \mathbf{else}\ stmt \\
&\mid \mathbf{if}\ expr\ \mathbf{then}\ stmt
\end{aligned}
$$

当看见输入记号 **if** 时，不能马上确定用哪个产生式来扩展 *stmt*。

一般来说，如果 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ 是 A 的两个产生式，输入串的前缀是从 α 推导出的非空串时，则不知道是用 $\alpha\beta_1$ 还是用 $\alpha\beta_2$ 来扩展 A。但是可以通过先扩展 A 到 $\alpha A'$ 来推迟这个决定。然后，看完了从 α 推出的输入后，再扩展 A' 到 β_1 或 β_2。这就是**提左因子**，原来的产生式成为：

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$

例 3.7 对于悬空 **else** 的文法

stmt →**if** *expr* **then** *stmt* **else** *stmt*

| **if** *expr* **then** *stmt*

| **other**

提左因子后的文法成为

stmt →**if** *expr* **then** *stmt optional_else_ part*

| **other**

optional_else_ part →**else** *stmt*

| ε

这样,如果输入的第一个记号是 **if**, 那么扩展 *stmt* 到 **if** *expr* **then** *stmt optional_else_ part*,等到 **if** *expr* **then** *stmt* 都看见后,再决定扩展 *optional_else_ part* 到 **else** *stmt* 还是到 ε。 □

*3.2.8 非上下文无关的语言构造

在很多编程语言中,仅用上下文无关文法难以实现其中一些语言构造的规范。本节将给出一些这样的构造,并用简单的抽象语言来说明其中的困难。

例 3.8 考虑抽象语言 $L_1 = \{wcw \mid w \text{ 属于} (a \mid b)^*\}$。$L_1$的句子的特点是,其前后是由 a 和 b 组成的相同的串,中间由 c 把它们隔开,例如 *aabcaab*。这个抽象语言是对程序中标识符的声明应先于其引用的抽象,*wcw* 中的第一个 w 代表标识符 w 的声明,第二个代表它的引用。可以证明该语言不是上下文无关语言,但是这个证明超出了本书的范围。这个例子意味着 C 和 Java 都不是上下文无关语言,因为它们都要求标识符的声明先于引用,并且允许标识符任意长。

由于这一点,描述这些语言语法的文法只是用 **id** 这样的记号来代表所有的标识符,而在这些语言的编译器中,由语义分析阶段检查标识符的声明必须先于引用。 □

例 3.9 语言 $L_2 = \{a^n b^m c^n d^m \mid n \geqslant 0, m \geqslant 0\}$ 不是上下文无关语言。L_2是正规式 $a^* b^* c^* d^*$ 所表示语言的子集,其要求是 a 和 c 的个数相等,b 和 d 的个数相等。它是对过程声明中形参个数和该过程引用的实参个数应该相同这一问题的抽象,a^n和 b^m代表两个过程声明的形参表中分别有 n 和 m 个参数,c^n和 d^m分别代表这两个过程调用的实参表。

语言中过程声明和引用的语法并不涉及参数的个数。例如 FORTRAN 的 call 语句可描述为:

stmt→ **call id** (*expr_list*)

expr_list →*expr_list*, *expr*

| *expr*

实参和形参个数的一致性检查也是放在语义分析阶段完成的。 □